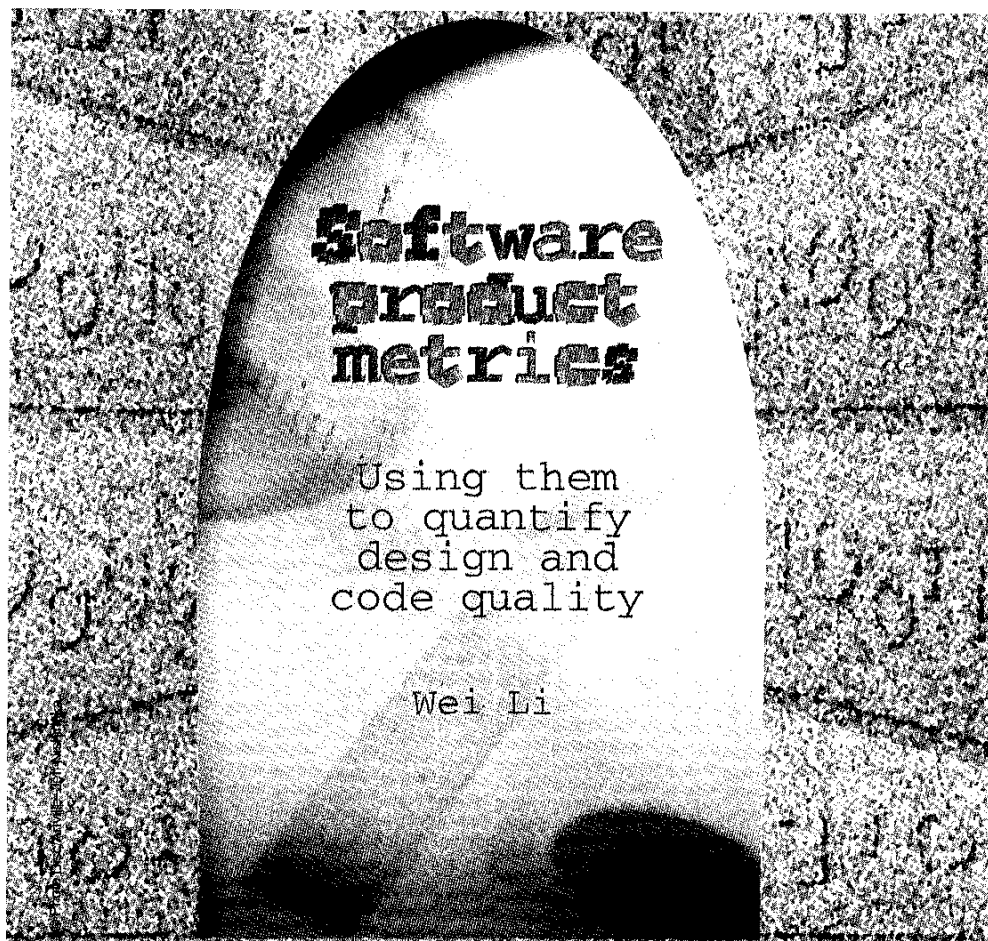




depends on the paradigm of the programming language used to implement the system. Two popular programming paradigms—procedural and object-oriented—are widely used in software industry. Programming languages such as Pascal and C support procedural paradigm. SmallTalk and JAVA characterize the object-oriented paradigm.

The software's internal structure can be quite different in the two programming paradigms. For example, functions and the interactions among the functions characterize a system constructed using the popular C programming language. SmallTalk or JAVA create their systems using classes, objects and interacting objects.

Software metrics that measure software products in different paradigms are also different. In the procedural paradigm, metrics measure functions and how functions interact. Classes and how classes interact are measured in the object-oriented paradigm. In the procedural paradigm, a function's signature (the name, type, and parameter list)—as well as how the functions interact with one another via function calls—make the structure of software.

In the object-oriented paradigm, a class' data and operation attributes and how classes are coupled with one another make the static structure of software. (An object-oriented software system may also be modeled by objects and interacting objects, which characterize the dynamic structure of the system. The dynamic modeling of

Software metrics are measures of software. Their primary use is to help us plan and predict software development. We can better control software quality and development effort if we can quantify the software. This is especially true in the early stages of development. Research has been done in the area of predicting software maintenance effort from software metrics. For example, Li and Henry successfully predicted maintenance effort measured from a collection of object-oriented (OO) metrics in 1993 using two Classic-Ada systems. Basili et al. predicted design quality in 1996 from the same OO metrics in 1996 using graduate student projects in C++. Our hope is that someday, software development will be as scientific as established science and engineering disciplines.

In general, software metrics can be classified into two categories: software product metrics and software process metrics. Software product metrics are measures of software products such as source code and design documents. Software process metrics are measures

of software development process. For example, the size of the software is a measure of the software product itself, thus a product metrics. The effort required to design a software system may be influenced by how the software is designed, thus a process metrics. In this article, only software product metrics will be addressed.

A software system, just like any other system, has an internal design structure and an external behavior. For example, the look and feel of a word processor is the software's external behavior. Internally, the software is designed and constructed by putting together software modules: the software's internal design structure.

The design of a car's engine, transmission, driving chassis, as well as the way these parts are put together influence the quality and behavior of the car. Similarly, the design of modules and the way these modules are put together influence the quality of the software. If we can properly measure the software's design structure, we can then better control the software product's quality.

The design structure of software

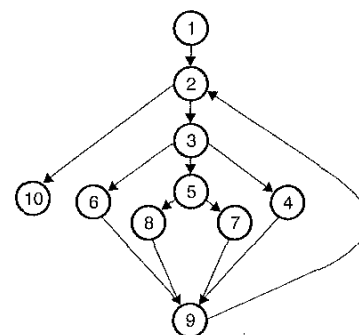


Fig. 1 Flow graph of main().

Table 1 Summary of the metrics for the Listing 1

Functions	Line of Code	Software metrics Information Flow Complexity	Cyclomatic Complexity
main()	20	256	6
push()	6	36	2
pop()	5	4	1
printStack()	7	64	3
isEmpty()	1	4	1
instructions()	1	4	1

object-oriented systems is beyond the scope of this article, however.)

Measuring in the procedural paradigm

There are two aspects in software structures in the procedural paradigm. The first one is the control flow within each function or procedures. (From now on, the term functions will be used to generally denote "functions and procedures" in the procedural paradigm.) The second one is the interactions among the functions.

The metrics measuring procedural structures include: the measurement of a function's size, the control flow complexity within a function and the information flow complexity.

The size of a function is a controversial but yet widely used metric. It is controversial because there is no perfect size measure that everyone agrees upon. Although variations of size measures exist, a common definition of size is the Lines of Code (LOC). LOC is calculated as the total number of source lines of code in a function with or without blank and comment lines; the decision to count or discount blank and comment lines is up to practitioners and a particular context. Usually, it does not matter so much either way as long as you use one definition consistently.

colons in a C function. The LOC metric for each function in the example is summarized in Table 1.

In 1976, McCabe proposed the Cyclomatic Complexity (CC) metric to measure the control flow complexity within a function. The metric is calculated by counting the number of enclosed regions plus one in the simplified flow chart of a function.

tently.

Listing 1 has a sample C program that contains several functions. The LOC metric is calculated as the total number of source lines of code excluding blank and comment lines; it is counted as the number of executable semi-

Figure 1 shows a simplified flow chart for the "main" function in Listing 1. Each node is numbered to match its corresponding block of source code in the function "main." There are four enclosed regions, therefore, the CC metric for "main" is five. A function's CC metric can also be calculated by two other means: 1) the number of single predicate in the function, or 2) $CC = E - N + 2$ where E is the number of edges and N is the number of nodes in the function's simplified flow chart.

This metric gives an indication of how complicated the control flow is in a function. It also indicates how many test cases are needed to perform the basis-path testing on the function. A summary of the CC metric values for all the functions in Listing 1 is in Table 1.

Both size and CC metrics are measures within a function. However, the size metric can be extended to measure a system. This is done by summing the LOC metric values of all the functions in the system. The Henry-Kafura Infor-

```
/* dynamic stack program */
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct stackNode { /* self-referential structure */
    int data;
    struct stackNode *nextPtr;
};
```

```
typedef struct stackNode STACKNODE;
typedef STACKNODE *STACKNODEPTR;
```

```
void push(STACKNODEPTR *, int);
int pop(STACKNODEPTR *);
int isEmpty(STACKNODEPTR);
void printStack(STACKNODEPTR);
void instructions(void);
```

```
main()
```

```
{
    STACKNODEPTR stackPtr = NULL; /* points to the stack top */
    int choice, value;

    1 {
        instructions();
        printf("? ");
        scanf("%d", &choice);

        2 while (choice != 3) {

            3 switch (choice) {

                4 case 1: /* push value onto stack */
                    printf("Enter an integer: ");
                    scanf("%d", &value);
                    push(&stackPtr, value);
                    printStack(stackPtr);
                    break;

                    5 case 2: /* pop value off stack */
                        if (!isEmpty(stackPtr))
                            pop(&stackPtr);
                        printf("The popped value is %d.\n",
                            pop(&stackPtr));

                7 default:
                    printStack(stackPtr);
                    break;

                6 case 3:
                    printf("Invalid choice.\n");
                    instructions();
                    break;

            }

            9 printf("? ");
            scanf("%d", &choice);

            10 printf("End of run.\n");
            return 0;
        }
    }
}
```

```
/* Print the instructions */
void instructions(void)
```

```
{
    printf("Enter choice:\n");
    printf("1 to push a value on the stack\n");
    printf("2 to pop a value off the stack\n");
    printf("3 to end program\n");
}
```

```
/* Insert a node at the stack top */
void push(STACKNODEPTR *topPtr, int info)
```

```
{
    STACKNODEPTR newPtr;
    newPtr = malloc(sizeof(STACKNODE));
    if (newPtr != NULL) {
        newPtr->data = info;
        newPtr->nextPtr = *topPtr;
        *topPtr = newPtr;
    }
    else
        printf("%d not inserted. No memory available.\n", info);
}
```

```
/* remove a node from the stack top */
int pop(STACKNODEPTR *topPtr)
```

```
{
    STACKNODEPTR tempPtr;
    int popValue;
    tempPtr = *topPtr;
    popValue = (*topPtr)->data;
    *topPtr = (*topPtr)->nextPtr;
    free(tempPtr);
    return popValue;
}
```

```
/* Print the stack */
```

```
void printStack(STACKNODEPTR currentPtr)
{
    if (currentPtr == NULL)
        printf("The stack is empty.\n");
    else {
        printf("The stack is:\n");
        while (currentPtr != NULL) {
            printf("%d -> ", currentPtr->data);
            currentPtr = currentPtr->nextPtr;
        }
    }
}
```

```
printf("NULL\n");
```

```
}
```

```
}
```

```
/* Is the stack empty? */
int isEmpty(STACKNODEPTR topPtr)
```

```
{
    return topPtr == NULL;
}
```

Listing 1: A Program Example in C

mation Flow Complexity metric is based on the fan-in and fan-out of a function. This metric measures how much interaction a function has with its environment in the system.

A fan-in can be a direct call from another function, an retrieval from a global variable whose value is used to make a decision, or an indirect local flow. An indirect local flow results when a function calls another function and the called function returns a value to the calling routine. That calling routine then passes the return value to a second function. There is an indirect local flow from the first called function to the second called function.

A fan-out can be either a direct call to another function or a global variable that is modified by the function or an indirect local flow. As in the example, the first called function had an indirect local flow out of it and into the second called function. The information flow complexity metric is calculated as (fan-in + fan-out)2. Table 1 shows the information flow complexity of all functions in Listing 1. Please note that the numbers include C library function calls.

Measuring in the object-oriented paradigm

Object-oriented (OO) systems have different structures than their procedural counterparts. This is because the two paradigms have different building blocks: functions for the procedural and classes for the OO paradigm.

```
// Class Location describes screen locations in X and Y coordinates.
// Class Point describes whether a point is hidden or visible.
// Class Circle describes the radius of a circle around a point.
```

```
enum Boolean { false, true};
```

```
class Location {
protected:
    int X;
    int Y;
public:
    Location(int InitX, int InitY) {X = InitX; Y = InitY;}
    int GetX() {return X;}
    int GetY() {return Y;}
};
```

```
class Point : public Location {
protected:
    Boolean Visible;
public:
    Point(int InitX, int InitY);
    virtual void Show(); // Show and Hide are virtual
    virtual void Hide();
    virtual void Drag(int DragBy); // new virtual drag function
    Boolean isVisible() {return Visible;}
    void MoveTo(int NewX, int NewY);
};
```

```
class Circle : public Point { // Derived from class Point and
// ultimately from class Location
```

```
protected:
    int Radius;
public:
    Circle(int InitX, int InitY, int InitRadius);
    void Show();
    void Hide();
    void Expand(int ExpandBy);
    void Contract(int ContractBy);
};
```

```
// member functions
```

```
Point::Point(int InitX, int InitY) : Location (InitX, InitY)
{
    Visible = false; // make invisible by default
}
```

```
void Point::Show()
{
    Visible = true;
    putpixel(X, Y, getcolor()); // uses default color
}
```

```
void Point::Hide()
{
    Visible = false;
    putpixel(X, Y, getbkcolor()); // uses background color to erase
}
```

Listing 2: A C++ example

The metrics that measure the object-oriented structure are also different. Chidamber and Kemerer proposed a suite of six metrics to measure OO design structure. They are: 1) Depth of Inheritance Tree (DIT), 2) Number of Children (NOC), 3) Weighted Method of a Class (WMC), 4) Response For a Class (RFC), 5) Coupling Between Objects (CBO), and 6) Lack of Cohesion Of Methods (LCOM).

```
void Point::MoveTo(int NewX, int NewY)
{
    Hide(); // make current point invisible
    X = NewX; // change X and Y coordinates to new location
    Y = NewY;
    Show(); // show point at new location
}
```

```
void Point::Drag(int DragBy)
{
    int DeltaX, DeltaY;
    int Fig.X, Fig.Y;
```

```
Show(); // display Fig. to be dragged
Fig.X = GetX(); // get initial position of Fig.
Fig.Y = GetY();
```

```
while (GetDelta(DeltaX, DeltaY)) // This is the drag loop
{
    // Apply delta to Fig. at X, Y
    Fig.X += (DeltaX * DragBy);
    Fig.Y += (DeltaY * DragBy);
    MoveTo(Fig.X, Fig.Y); // tell Fig. to move
};
}
```

```
Circle::Circle(int InitX, int InitY, int InitRadius) : Point (InitX, InitY)
{
    Radius = InitRadius;
}
```

```
void Circle::Show()
{
    Visible = true;
    circle(X, Y, Radius); // draw the circle
}
```

```
void Circle::Hide()
{
    unsigned int TempColor; // to save current color
    TempColor = getcolor(); // set to current color
    setcolor(getbkcolor()); // set drawing color to background
    Visible = false;
    circle(X, Y, Radius); // draw in background color to
    setcolor(TempColor); // set color back to current color
}
```

```
void Circle::Expand(int ExpandBy)
{
    Hide(); // erase old circle
    Radius += ExpandBy; // expand radius
    if (Radius < 0) // avoid negative radius
        Radius = 0;
    Show(); // draw new circle
}
```

```
void Circle::Contract(int ContractBy)
{
    Expand(-ContractBy); // redraws with (Radius-ContractBy)
}
```

```
class A {
public:
    int a;
    void A();
};

class B {
public:
    A aaA;
    void B();
};
```

Fig. 2 An example of coupling through abstract data type

Table 2 The metric values for location, point, and circle classes

	CTA	CTM	LCOM	NAC	NDC	NLM	WMC
Location	0	0	1	0	2	3	3
Point	0	3	3	1	1	6	6
Circle	0	6	2	2	0	5	6

Decedent Class (NDC) states how many decedent classes the class has in the same hierarchy. The Number of Local Methods (NLM) metrics gives an indication of how large the class' local interface is. The Class Method Complexity (CMC) measures the combined complexity of all local methods in a class.

The Coupling Through Abstract data types (CTA) measures how many classes the class uses as abstract data types in its data attribute declaration. The Coupling Through Message passing (CTM) metric measures how many messages an object from the class can send out when executing. Listing 2 has a C++ sample program that contains three classes. Table 2 summarizes the OO metrics for all the classes in Listing 2.

The NAC metric is intended to measure how many classes may potentially affect the design of the class because of inheritance. The NDC metric is designed to measure how many decedent classes the class may affect because of inheritance. Inheritance helps reuse of already designed classes when designing a new class. But, it also introduces coupling among the classes. If a change is made in an ancestor class, the change could potentially affect all its descendent classes. Therefore, the NAC and NDC metrics are an important in measuring potential change propagation through the inheritance hierarchy. Both NAC and NDC metrics carry the unit of "class."

The NLM metric is designed to measure the size of a class' local interface. A class provides a form of encapsulation for it's data attributes. (In the sense, that all accesses to the data attributes are regulated through the locally defined or inherited public methods—the interface of the class.) The public methods defined locally in the class gives an indication of how large the interface the class provides without influence from inheritance. The NLM metric has the unit of "method."

A class can be coupled with other classes in three different forms: inheritance, abstract data type and message passing. When a class inherits from another class, directly or indirectly, the two

classes are coupled. This type of class coupling breaks the encapsulation that OO programming provides. The NAC and NDC metrics, or some composite metrics made of the NAC and the NDC metrics, can capture this attribute well.

Two classes are coupled when one class uses the other class as an abstract data type. Consider the example in Fig. 2. Class B is coupled with class A through the use of abstract data type. This is because class B uses class A in its data-attribute declaration. The CTA metric is intended to measure this form of class coupling.

Two classes can be coupled because one class sends a message to an object of another class, without involving the two classes through inheritance or abstract data types. Consider the example in Fig. 3. Both classes A and B are in the same OO design. Also, they are not related through inheritance or abstract data type as a class attribute. However, class A is coupled with class B because one of A's methods sends a message to B's object.

For a class, there are two aspects to message-passing coupling. One aspect is the scale to which the class can receive messages from other classes. The other aspect is the scope to which the class sends out messages to other classes. Some combinations of the NLM and the NAC metrics can measure the scope to which a class receives messages. Therefore, the discussion of message-passing coupling is limited to the scope to which the class sends out messages to other classes. The Coupling Through Message Passing (CTM) metric is designed to measure this type of class.

Summary

Software metrics is a relatively young science and engineering discipline. It will take sometime before the field matures to the level of other established disciplines. The metrics that are discussed in this article are representative enough to show the basics in this area. A chal-

lenging task facing software metrics research is validation of the metrics. If we want to use metrics in industrial software development effectively, they must be meaningful and useful. It is not difficult to propose some quantitative measures for software. It is not easy, however, to validate them.

Acknowledgments

I would like to thank Dr. Letha Etzkorn, Mr. Amirali Jalia and Mr. Mohammad Alshayeb for their assistance.

Read more about it

- Basili, V. L., Briand, L., and Melo, W. L., "A Validation of Object-Oriented Metrics as Quality Indicators," *IEEE Trans. Software Eng.* 10, 751-761 (1996).
- Chidamber, S. R. and Kemerer, C. F., "A Metrics Suite for Object Oriented Design," *IEEE Trans. Software Eng.* 6, 476-493 (1994).
- Deitel, H.M. and Deitel, P.J. *C How to Program*, Prentice Hall, New Jersey (1996).
- Henry, S. and D. Kafura (1981) "Software Structure Metrics Based on Information Flow," *IEEE Transaction on Software Engineering*, SE-7(5), pp. 510-518.
- Li, W., *Applying Software Maintenance Metrics in the Object-Oriented Software Development Life Cycle*, Ph.D. Dissertation, Virginia Polytechnic Institute and State University, Blacksburg, Virginia, 1992.
- Li, W. and Henry, S., "Object-Oriented Metrics Which Predict Maintainability," *J. Systems and Software* 2, 111-122 (1993).
- McCabe, T., "A Complexity Measure," *IEEE Trans. Software Eng.* 2, 308 - 320 (1976).
- Pressman, R., *Software Engineering*, McGraw Hill, New York (1997).

About the author

Wei Li received a Ph.D. in Computer Science from Virginia Tech. He is currently an assistant professor at the University of Alabama in Huntsville. His areas of interest are object-oriented programming, software metrics, and reusable components. Dr. Li is a member of the ACM and IEEE Computer Society.

```

class A {
    int a;
public:
    void A();
    void A1 (B* b) { b->B1(); }
};

class B {
    A anA;
public:
    void B();
    void B1 ();
};

```

Fig. 3 An example of coupling through message passing